



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 · Design & Analysis of Algorithms · Fall 2026

Big-O — The Intuition

Three moves, six shapes, and why a faster machine cannot fix a bad one

Session 3 of 24 · Friday 25 September 2026 · 8:30 – 9:50 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 3 OF 24

Big-O — The Intuition

Friday, 25 September 2026 · 8:30 – 9:50 AM

LEARNING OBJECTIVES

- ✓ Explain why constants and lower-order terms are discarded when describing cost.
- ✓ Apply the three simplification moves to a counted running time.
- ✓ Name the six growth classes and recognise each one from the structure of Python code.
- ✓ Explain why the growth rate of an algorithm outranks the speed of the machine it runs on.

FROM SESSION 2

We counted comparisons and timed both searches. Linear search multiplies when n doubles; binary search adds one. Today we give these behaviours names, so that we can talk about them without a table of numbers every time.

1 · Why a Notation Is Needed 15 min

Suppose careful counting gives a function the running time

$$T(n) = 3n^2 + 17n + 42$$

This is more precision than anyone can use. The 3 depends on how fast your processor multiplies; the 17 on how your compiler arranged the loop; the 42 on the cost of calling a function. Change the machine and all three numbers change — but something about the expression does not change. Let us find out what.

n	$3n^2$	$17n$	42	share carried by $3n^2$
1	3	17	42	5%
10	300	170	42	59%
100	30,000	1,700	42	94.5%
10,000	300,000,000	170,000	42	99.94%

READ THE LAST COLUMN DOWNWARD

At $n = 1$ the constant 42 is most of the cost — and nobody cares, because everything is fast at $n = 1$. By $n = 10,000$ the other two terms together account for less than one part in a thousand. In the region where performance is actually a problem, the expression is its leading term.

2 • The Three Moves 20 min

Move 1 — Drop the constants

$5n$, n and $0.01n$ all become n . A constant factor is what you buy with a faster processor. This course is about the factors you cannot buy.

Move 2 — Keep the dominant term

In a sum, the fastest-growing term wins eventually and by an ever-widening margin. $n^2 + 1000n$ becomes n^2 , even though $1000n$ is larger until $n = 1000$.

Move 3 — Think large n

The description is about behaviour *eventually*. It says nothing about $n = 5$, and it is not supposed to.

What survives

A statement about **shape** that outlives the machine, the compiler, the language and the decade. We write it as $O(n^2)$, pronounced “big oh of n squared”.

$$T(n) = 3n^2 + 17n + 42 \text{ is } O(n^2)$$

2.1 Practising the moves

Counted running time	After Move 1	After Move 2	Written as
$7n + 3$	$n + 3$	n	$O(n)$
$n^2/2 - n/2$	$n^2 - n$	n^2	$O(n^2)$
100	1	1	$O(1)$
$4n \log n + 90n$	$n \log n + n$	$n \log n$	$O(n \log n)$
$2^n + n^{100}$	$2^n + n^{100}$	2^n	$O(2^n)$

The last row surprises people. n^{100} is astronomically larger than 2^n for every n below about 1000 — but exponentials overtake every polynomial eventually, and “eventually” is what Move 3 tells us to look at.

What the moves are not

They are not a claim that constants do not matter in practice. A program twice as fast is twice as fast, and your users will notice. The claim is narrower and stronger: constants cannot change which algorithm wins *in the end*, and they are not a property of the algorithm at all.

3 · Six Shapes, Read Off the Code 30 min

In practice you will rarely calculate $T(n)$ and simplify it. You will read the growth class off the structure of the code. Six shapes cover nearly everything you meet this semester.

PY CONSTANT

```
def first_element(A):
    return A[0]
```

$O(1)$ The work does not depend on `len(A)`. Double the input and nothing changes. Indexing, arithmetic and dictionary lookup all live here.

PY LINEAR

```
def total(A):
    s = 0
    for v in A:
        s += v
    return s
```

$O(n)$

One pass, constant work per element. Double the input, double the time. Any single loop over the data

lands here — and so does linear search.

PY LOGARITHMIC

```
def count_halvings(n):
    steps = 0
    while n > 1:
        n = n // 2      # divided, not decremented
        steps += 1
    return steps
```

 $O(\log n)$ Double the input and the cost rises by exactly **one**. This is binary search. The operator is whatdecides: `n -= 1` would make this linear, `n //= 2` makes it logarithmic.**PY** QUADRATIC

```
def count_pairs(A):
    pairs = 0
    for i in range(len(A)):
        for j in range(len(A)):
            if A[i] + A[j] == 0:
                pairs += 1
    return pairs
```

 $O(n^2)$ A loop inside a loop, each over the whole input: $n \times n$. Double the input and the time goes up **fourfold**.

This is the first class that genuinely fails on real data — around a million elements it becomes unusable.

PY LINEARITHMIC

```
def biggest_gap(A):
    B = sorted(A)          # sorting costs  $O(n \log n)$ 
    best = 0
    for i in range(len(B) - 1):  # one extra pass costs  $O(n)$ 
        best = max(best, B[i + 1] - B[i])
    return best
```

 $O(n \log n)$ Sorting dominates the scan, so $O(n \log n) + O(n)$ collapses to $O(n \log n)$ by Move 2. This is the

cost of every good sorting algorithm and, in practice, the ceiling of what is comfortable on large data.

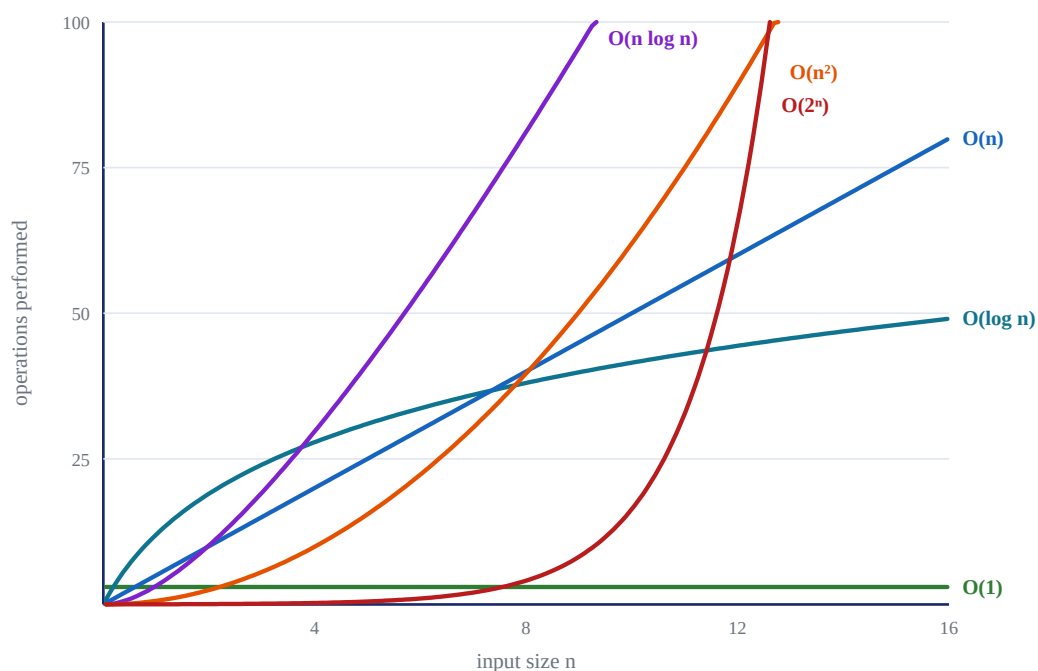
PY EXPONENTIAL

```
def subsets(A):
    if not A:
        return [[]]
    rest = subsets(A[1:])
    return rest + [[A[0]] + s for s in rest]
```

$O(2^n)$

Every extra element **doubles** the answer and therefore the time. At $n = 50$ there are about 10^{15} subsets: a machine producing a billion per second needs roughly thirteen days. At $n = 100$ it needs longer than the universe has existed.

3.1 Reading the shapes side by side



The gallery. All six curves on one pair of axes for inputs up to $n = 16$. Notice how meaningless the ranking is on the far left — at $n = 3$ the exponential curve is the cheapest of them all. Big-O deliberately says nothing about that region. By $n = 12$ the ordering is permanent, and no constant factor will ever reverse it.

Class	Double $n \Rightarrow$ time is ...	Where it comes from	Practical feel
$O(1)$	unchanged	indexing, arithmetic, hash lookup	free
$O(\log n)$	one step more	repeated halving	effectively free
$O(n)$	twice as long	one pass over the data	fair
$O(n \log n)$	a little more than twice	sorting, divide and conquer	the practical ceiling
$O(n^2)$	four times as long	nested loop over the same data	fails near a million

$O(2^n)$

squared

trying every subset

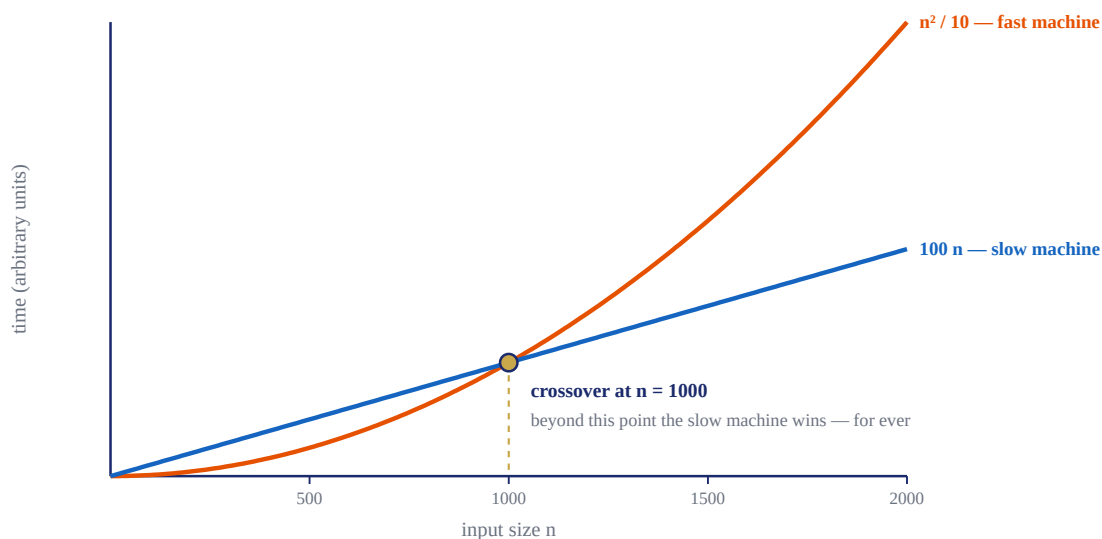
fails near fifty

Compare this table with the doubling table from Session 2. They are the same table. What was an observation about measurements has become a classification of algorithms.

4 · Why Shape Beats Hardware

15 min

The natural objection: if my computer is fast enough, does the growth class really matter? Suppose you run an $O(n^2)$ program on a machine ten times faster than mine, while I run an $O(n)$ program on an old laptop. Your cost is $n^2/10$; mine is $100n$.



Hardware buys a crossover point, not a victory. Your faster machine wins for every input below $n = 1000$. From there on mine wins, and the gap widens without limit. Buying a machine a hundred times faster moves the crossing to $n = 10,000$ — it does not change who wins in the end.

THE CENTRAL CLAIM OF THIS COURSE

Hardware improves by constant factors. Algorithms improve by changing the shape. Constant factors are bought; shapes are earned. Over the last fifty years, the speed-ups obtained from better algorithms have exceeded those from faster processors on many of the problems that matter most.

A caveat worth stating out loud

None of this says the $O(n^2)$ program is the wrong choice. If you know n will never exceed 200, the quadratic program may be simpler, shorter, and faster in practice. Knowing which regime you are in is part of engineering judgement; Big-O tells you what happens when you are wrong about n .

SESSION 3 TAKEAWAY

Drop the constants, keep the dominant term, look at large n — what remains is the **shape** of the cost, and that is what Big-O records. Six shapes cover most of computing, and you can usually read the right one straight off the loop structure. Hardware shifts the crossover point; only a better shape changes the outcome.

IN-CLASS EXERCISES (work in pairs)

1. Simplify to a single growth class: $4n + 9$; $n^2/2 - 3n$; $2n \log n + 5000n$; 7 ; $n + 2^n$; $n^2 + n \log n + 1000$.
2. A program takes 3 seconds on input of size 1000 and 12 seconds on size 2000. Which class is it in? Predict the time for size 4000 and for size 16,000.
3. Give the growth class of each, with a one-line reason: (a) finding the largest element of a list; (b) checking whether a list is sorted; (c) checking whether any two elements are equal, using two nested loops; (d) printing all the ways to arrange n people in a queue.
4. Your colleague says “my algorithm is $O(n^2)$, but the constant is tiny, so it beats the $O(n \log n)$ one.” Under what circumstances is this true? Under what circumstances is it a dangerous thing to say?
5. Sketch, without a computer, roughly where $n \log n$ and $n^2/4$ cross.

Homework · due Tuesday 29 September

1. For each of the six functions in §3, run the doubling test at $n = 500, 1000, 2000, 4000$ (use smaller sizes for `subsets` !) and confirm that the measured reaction matches the stated class. Submit code and tables.
2. Write down $T(n)$ exactly for `count_pairs` — the precise number of times the inner statement runs — then apply the three moves in writing, one at a time.
3. Find the smallest n greater than 1 for which 2^n exceeds n^5 , by writing a short loop. Then explain in two sentences why we still call 2^n the faster-growing of the two.
4. Take the `subsets` function and time it for $n = 10, 15, 20, 22$. Fit the reaction: by what factor does the time change when n increases by *one*? Does that match the claim in §3?
5. Reading: CLRS §3.1. Half a page, in your own words, on why asymptotic notation discards constant factors — and one sentence on when that is a bad idea.

Next session · Tuesday 29 September: The definition behind the intuition. What “ f is $O(g)$ ” actually asserts, how to prove it in three lines, and the two companions to Big-O — Ω and Θ — that let you say “at least” and

“exactly”.